

BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING

EEE-312
Digital Signal Processing Laboratory

Final Project Report
Section:C1 Group: 6

Detection Of Resistor and Calculate Its Value
by Detecting color band using Image Processing

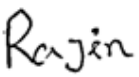
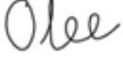
Course Instructors:

- 1.Shahed Ahmed, Lecturer, EEE, BUET
- 2.Md. Obaidur Rahman, Part-Time Lecturer, EEE, BUET

Signature of Instructor: _____

Academic Honesty Statement:

"In signing this statement, We hereby certify that the work on this project is our own and that we have not copied the work of any other students (past or present), and cited all relevant sources while completing this project. We understand that if we fail to honor this agreement, We will each receive a score of ZERO for this project and be subject to failure of this course."

Signature:  Full Name: Rifat Rahman Student ID: 2006137	Signature:  Full Name: Rajin Zeeshan Student ID: 2006142
Signature:  Full Name: Ayon Kumar Dey Student ID: 2006147	Signature:  Full Name: Ahnaf Rashid Olee Student ID: 2006158
Signature:  Full Name: Mushfiqur Rahman Student ID: 2006161	

ABSTRACT

In the realm of electronics, the accurate detection and calculation of resistor values are crucial for circuit design and troubleshooting. This project presents a novel approach utilizing image processing techniques for the automated detection and calculation of resistor values based on their color bands. By leveraging computer vision algorithms, the proposed system aims to streamline and enhance the process of resistor identification, reducing the reliance on manual interpretation and potential human error. The methodology involves capturing images of resistors using a digital camera or smartphone, followed by preprocessing steps to enhance image quality and isolate the resistor bands. Subsequently, color segmentation algorithms are employed to identify and extract the color bands from the resistor image. Machine learning techniques may also be incorporated to classify the colors accurately, thereby facilitating the determination of the resistor value. The system's effectiveness is evaluated through experimental validation, comparing the calculated resistor values against manually measured values for various resistors. Additionally, the performance of the image processing algorithms in different lighting conditions and resistor orientations is assessed to ascertain the system's robustness and reliability. Overall, the proposed approach demonstrates promising results in the accurate detection and calculation of resistor values through image processing techniques, offering a convenient and efficient solution for electronics enthusiasts, technicians, and engineers alike. This project contributes to advancing automation in electronics assembly and circuit analysis, paving the way for further innovations in image-based component recognition and characterization.

Table Of Content

ABSTRACT.....	i
1 INTRODUCTION:.....	iv
2 OBJECTIVE:	iv
2.1 System Formulation:.....	iv
2.2 Accuracy Measurement:.....	iv
2.3 Optimization:.....	v
3 Theory:	v
3.1 Morphological operation:.....	vi
3.Erosion	vi
2.Dilation:.....	vi
3.Opening:.....	vi
4.Closing:.....	vii
4 Procedure:.....	vii
4.1 Image segmentation:.....	vii
Converting Binary Form of Masked Image after Global Thresholding:	viii
• Converting Binary image to Inverted Image:.....	ix
• Morphological Operation to the Inverted Image:	ix
4.2 Resistor checking:	x
4.3 Cropping and detecting the resistors from other components:	xi
4.4 Detecting the Color Bands:.....	xii
4.5 Resistor orientation:	xiii
Code for checking orientation:	xiii
4.5.1 Horizontal flip:.....	xiii
4.5.2 Vertical flip:	xiv
4.6 Calculating Value of the Resistor using Different Band Values:	xiv
5 Features:	xv
5.1 Extracting Resistors:	xv
5.2 Finding Resistor bands:.....	xv
5.3 Orientation:	xv
5.4 Resistor Value:	xv
6 Result:	xv
7 User Manual.....	xvi
8 Novelty:.....	xvi
9 Social Impact:	xvii

9.1	Accessibility:	xvii
9.2	Education:.....	xvii
9.3	Efficiency:.....	xvii
9.4	Innovation:.....	xvii
9.5	Environmental impact:	xvii
10	Individual Contribution of Each Member	xvii
11	Limitations:	xviii
11.1	Accuracy	xviii
11.2	Variability in Resistor Type:	xviii
11.3	Complexity of Image Processing Algorithms:	xviii
11.4	Dependency on Camera Quality.....	xviii
11.5	Calibration and Maintenance:	xviii
11.6	Limited resistor Field:	xviii
12	FUTURE WORK:	xix
12.1	Enhanced Color Detection Algorithms:	xix
12.2	Deep Learning Approaches:	xix
12.3	Noise Reduction Techniques:	xix
12.4	Integration with Mobile Applications:.....	xix
12.5	User Interface Improvements:	xix
12.6	Accessibility Features:	xix
13	References:	xx

1 INTRODUCTION:

In the domain of electronics engineering, the precise identification and calculation of resistor values constitute fundamental tasks pivotal for the successful design, implementation, and troubleshooting of electronic circuits. Traditionally, manual methods involving visual inspection of color bands have been employed for resistor characterization, presenting challenges associated with subjectivity, human error, and time consumption. In response to these limitations, the utilization of image processing techniques has emerged as a promising avenue for automating resistor detection and value calculation, offering improved accuracy, efficiency, and objectivity. This paper delves into the development and implementation of a novel approach aimed at enhancing the process of resistor identification and value determination through the utilization of image processing methodologies. By harnessing the capabilities of computer vision algorithms, the proposed system endeavors to revolutionize the conventional methods by enabling automated detection and analysis of resistor color bands, thus mitigating the reliance on manual interpretation, minimizing the associated risks of error. This project sets the stage for a comprehensive exploration of the proposed approach, encompassing its theoretical foundations, implementation details, experimental validation, and potential applications within the broader context of electronics engineering and automation. In the domain of electronics engineering, the precise identification and calculation of resistor values constitute fundamental tasks pivotal for the successful design, implementation, and troubleshooting of electronic circuits.

2 OBJECTIVE:

2.1 System Formulation:

The whole project is done using MATLAB to produce a faster solution for calculating values of multiple resistors, regardless of how they are placed. Our project emphasizes taking resistor pictures as inputs and calculates the values of multiple given resistors. Our motive was to use MATLAB programs to design and integrate color segmentation algorithms to accurately identify and extract color bands from resistor images using image processing techniques. We will Implement algorithms for preprocessing digital images to enhance quality and isolate resistor color bands effectively.

2.2 Accuracy Measurement:

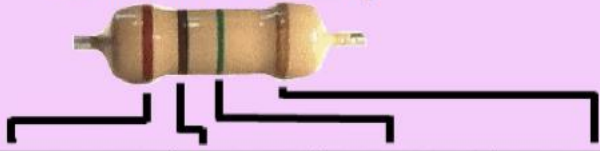
Accuracy of the program depends on factors like brightness, contrast, hue, saturation, orientation of resistors (Horizontal orientation or vertical, flipped, or non-flipped), number of components (no resistor or multiple resistors) etc. We have thought of all the glitches and developed our codes to solve the problems. Even the orientation of the resistors will not be a problem with our developed code. Then we will evaluate the system's performance through experimental validation, comparing calculated resistor values against manually measured values across a range of resistors. With this, we can assess the robustness and reliability of the image processing algorithms under various lighting conditions and resistor orientations.

2.3 Optimization:

We will optimize the system for efficiency and accuracy in detecting and calculating resistor values, aiming for a high degree of automation and minimal human intervention. We will provide a user-friendly interface for easy interaction with the system, enabling electronics enthusiasts, technicians, and engineers to utilize the tool effectively. The aim of our project is to contribute to advancing automation in electronics assembly and circuit analysis through innovative image-based component recognition and characterization techniques.

3 Theory:

Resistors have assorted color bands which are used to identify their values. Any resistor may have 4 color bands including a tolerance band. These bands and their sequences are used for calculating the values of a particular resistor. If we consider the following picture, we will be able to get a better idea.

4 Band Resistor Color Coding				
				
COLOR	1ST BAND	2ND BAND	MULTIPLIER	TOLERANCE
BLACK	0	0	x1Ω	
BROWN	1	1	x10Ω	±1%
RED	2	2	x100Ω	±2%
ORANGE	3	3	x1000Ω	
YELLOW	4	4	x10000Ω	
GREEN	5	5	x100000Ω	±0.5%
BLUE	6	6	x1000000Ω	±0.25%
VIOLET	7	7	x10000000Ω	±0.1%
GREY	8	8		±0.05%
WHITE	9	9		
GOLD			0.1	±5%
SILVER			0.01	±10%

The tolerance band of resistors are often placed on one end and is kept at a distance from the other color bands. The tolerance band is kept for adding or subtracting a percentage of error from the values calculated. The rest of the color bands are used for calculating the values. Each color is given a specific number for a specific band. The specific color of each band gives us some specific numbers. We can simply understand how to use these numbers by using a simple method.

Band 1, Band 2, Band 3 = Color, Color x 10 color in Ohm (Ω)

Now if we take a resistor shown in the image, we see that the color band sequence is red, purple, and brown.

If we put these values in the equation, we will get the desired value of the resistor

(tolerance excluded).

Band 1, Band 2, Band 3= Brown, Black x 10Red in Ohm (Ω) = 10×10^2 Ohm (Ω)

$$= 1000 \text{ Ohm } (\Omega)$$

And $270 \pm 5\%$ Ohm (Ω) for the tolerance.

3.1 Morphological operation:

Morphological operations are essential techniques within the realm of image processing, aimed at modifying the structure and shape of objects present in an image. They achieve this by employing a predefined structuring element, often represented as a small matrix or kernel, which serves as a template for altering the pixels within the image. This alteration process is contingent upon the neighborhood surrounding each pixel, as determined by the dimensions and arrangement of the structuring element. These operations play a pivotal role in various image processing tasks, including noise reduction, where they can help eliminate unwanted artifacts and disturbances present in the image. Additionally, morphological operations are instrumental in edge detection, allowing for the identification and delineation of boundaries between different regions or objects within the image. Furthermore, they facilitate object segmentation, enabling the separation and extraction of distinct entities or features present in the image, which is crucial for subsequent analysis and interpretation.

3.Erosion

- Erosion is a morphological operation that shrinks or erodes the boundaries of foreground objects in a binary image.
- It works by sliding a structuring element (a small matrix) over the image and replacing each pixel with the minimum pixel value within the neighborhood defined by the structuring element.
- Erosion is useful for removing small objects, separating overlapping objects, and smoothing object boundaries.

2.Dilation:

- Dilation is the opposite of erosion and is used to expand or dilate the boundaries of foreground objects in a binary image.
- Like erosion, it involves sliding a structuring element over the image, but instead of taking the minimum pixel value, it takes the maximum pixel value within the neighborhood defined by the structuring element.
- Dilation is useful for closing small gaps between objects, joining broken parts of objects, and increasing the size of objects.

3.Opening:

- Opening is a combination of erosion followed by dilation and is used to remove small objects and smooth object boundaries in binary images.
- It involves applying erosion first to remove small details, followed by dilation to restore the remaining object sizes.

- Opening helps in preprocessing tasks such as noise removal, separating touching objects, and improving object segmentation.

4.Closing:

- Closing is the opposite of opening and is used to fill small holes and gaps in binary images.
- It involves applying dilation followed by erosion, which helps in closing small breaks in object boundaries and joining nearby objects.
- Closing is useful for tasks such as completing object contours, filling gaps in text characters, and restoring the shape of objects after erosion.

These morphological operations are widely used in various image processing applications, including segmentation, feature extraction, noise reduction, and object recognition. They play a crucial role in enhancing and analyzing digital images for further processing or interpretation.

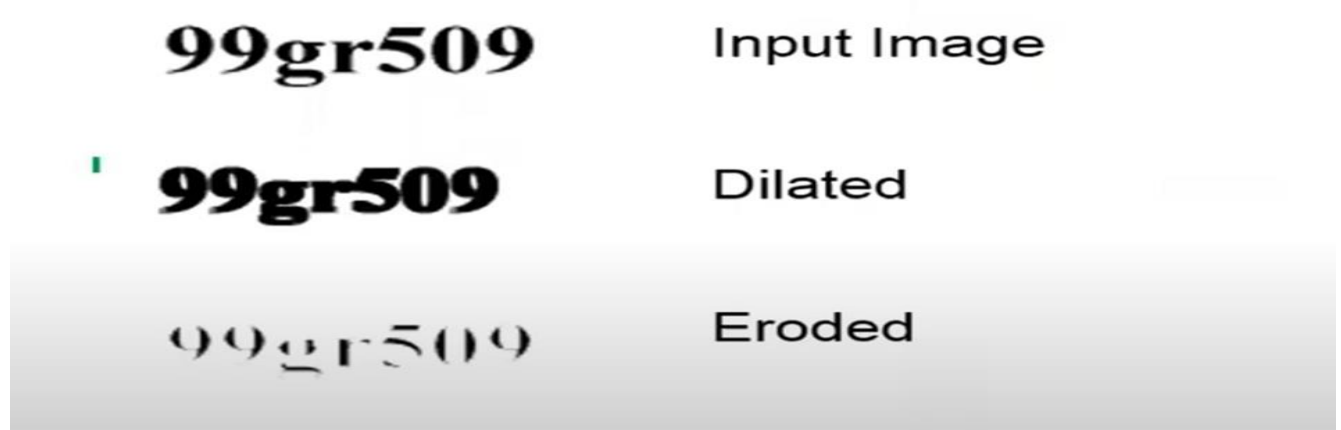


Figure 3: Morphological Operation

4 Procedure:

4.1 Image segmentation:

In this section different components of the main image will be segmentized to detect the resistor.

```
%% Input image
clc;
close all;
clear all;
Imain=imread('C:\Users\USER\Desktop\matlab\Resistor Images\res16clce.jpg');
Igray=rgb2gray(Imain);
imshow(Imain);title('main')
figure;
imshow(Igray);title('igray');
hold on;
```


This part of the code takes input from the user for segmentation of various parts of the image. Then converts the image to grayscale from RGB.

```
% Binary mask of main image
[BWmain,maskedimg] = segmentImageNmy(Igray);
figure;
imshow(BWmain);title('bw');
```

This portion of the code is used for segmenting a grayscale image Igray using a function called segmentImageNmy. The image segmenter app returns the binary form of the main image after performing morphological operation. The segmented image is stored in BWmain.

Converting Binary Form of Masked Image after Global Thresholding:

Global thresholding is a fundamental image processing technique used to create a binary image from a grayscale or color image. It aims to separate objects or regions of interest from the background by determining a single threshold value that distinguishes between foreground and background pixels.



Figure: Global Thresholding

After applying global thresholding, the image is segmented into foreground and background regions using two distinct colors. Subsequently, we will convert the segmented image into binary format, assigning the black color to represent the foreground and the white color to represent the background

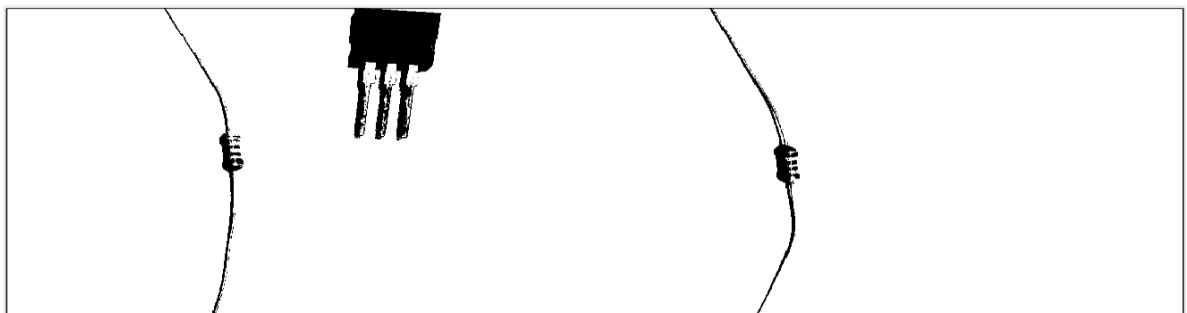


Figure: Binary Form of Masked Image

- **Converting Binary image to Inverted Image:** Now, we'll switch the background color to black and the foreground color to white.



Figure: Inverted Image

- **Morphological Operation to the Inverted Image:** In the inverted image, there are some black stains within the white area. We will now eliminate these black artifacts by applying morphological operations.



Figure: Morphological operation of the inverted image

```
%% structuring according to brightness
hsv=rgb2hsv(Imain);
v = hsv(:, :, 3);
avgV = mean(v(:));
```

Here the RGB image is converted to HSV representation to extract the value component of the main image.

```
%%
if avgV < .9
    se = strel('square',41);
    Iopenned = imopen(BWmain,se);
    % figure;
    % imshow(Iopenned);title('iopen');
elseif avgV > .9
    se = strel('square',18);
    Iopenned = imopen(BWmain,se);
    % figure;
    % imshow(Iopenned);
end
```

After getting the value component of the main image the average value is used to perform morphological opening on the binary image with different structuring elements based on the average value.

```

%% extracting features
Iregion = regionprops(Iopenned, 'centroid');
[labeled,numObjects] = bwlabel(Iopenned,4);
v=numObjects;
imshow(labeled);
figure;
imshow(label2rgb(labeled, 'jet', 'k', 'shuffle'));
title('Labeled Image');
stats = regionprops(labeled,'Eccentricity','Area','BoundingBox');
areas = [stats.Area];
eccentricities = [stats.Eccentricity];

idxOfResistors = find(eccentricities);
statsDefects = stats(idxOfResistors);

```

Here some key features like Eccentricity, Area, Bounding Box have been calculated for later use.

4.2 Resistor checking:

Resistor Checking: In this step, all the components will be checked to see if that is a resistor or not. We know that resistor has a unique yellowish brown body color with color bands on it. No other circuit component has this unique body color and so depending on this we shall check out for resistor among all the components existing in the image. We have created “masked yellow” function to check the color.



Fig: yellow background of resistor

Code for resistor checking:

```

%% detecting whether the component is a resistor or not
for idx = 1 : length(idxOfResistors)
    res{idx} = imcrop(Imain,statsDefects(idx).BoundingBox);
    figure;

    for i = 1:length(res)
        subplot(1, length(res), i); % Display each image in a separate subplot
        imshow(res{i}); % Display the i-th image
    end
end
m=1;
for i=1:length(res)
    check=0;
    [BWres,~]= ResCheckmy(res{i});

    % figure;
    % imshow(BWres);
    se = strel('disk',6);
    Iopennedres = imopen(BWres,se);
    Iregion = regionprops(Iopennedres, 'centroid');
    [labeled,numObjectsRes] = bwlabel(Iopennedres,4);
    if numObjectsRes~=0
        h = rectangle('Position',statsDefects(i).BoundingBox,'LineWidth',2);
        set(h,'EdgeColor',[0 0.75 0])
        hold on;
        % position=[statsDefects(i).BoundingBox(1,1)-50 statsDefects(i).BoundingBox(1,2)-50];
        position=[statsDefects(i).BoundingBox(1,1)+statsDefects(i).BoundingBox(1,3)-50 statsDefects(i).BoundingBox(1,2)-50];
        text('Position',position,'string',num2str(m));
        resimg{m} = imcrop(Imain,statsDefects(i).BoundingBox);
        figure;

        for j = 1:length(resimg)
            subplot(1, length(resimg), j); % Display each image in a separate subplot
            imshow(resimg{j}); % Display the i-th image
        end
    end
end

```

```

        m=m+1;
        check=1;
    end
    if check ~=1
        [BWres,~]= ResCheck2my(res{i});
        se = strel('disk',2);
        Iopennedres = imopen(BWres,se);
        Iregion = regionprops(Iopennedres, 'centroid');
        [labeled,numObjectsRes] = bwlabel(Iopennedres,4);
        if numObjectsRes~=0
            h = rectangle('Position',statsDefects(i).BoundingBox,'LineWidth',2);
            set(h,'EdgeColor',[0.75 0 0])
            % hold on;
            % position=[statsDefects(i).BoundingBox(1,1)-50 statsDefects(i).BoundingBox(1,2)-50];
            position=[statsDefects(i).BoundingBox(1,1)+statsDefects(i).BoundingBox(1,3)-50 statsDefects(i).BoundingBox(1,2)-50];
            text('Position',position,'string',num2str(m));
            resimg{m} = imcrop(Imain,statsDefects(i).BoundingBox);

            figure;
            for i = 1:length(resimg)
                subplot(1, length(resimg), i); % Display each image in a separate subplot
                imshow(resimg{i}); % Display the i-th image
            end

            m=m+1;
            check=1;
        end
    end
end

title(['There are ', num2str(length(resimg)), ' resistors out of ',num2str(length(res)), ' in the image!'],'color','b');

```

4.3 Cropping and detecting the resistors from other components:

Using the binary mask, images of each component are cropped. Because every resistor has a unique light brown color, we can use this to differentiate it from other electrical components. So, by using a mask, we detect the resistors and keep only the cropped images of resistors.

In this case, we used bounding box data to detect the resistor parts from original image and marked them with red outlined box.

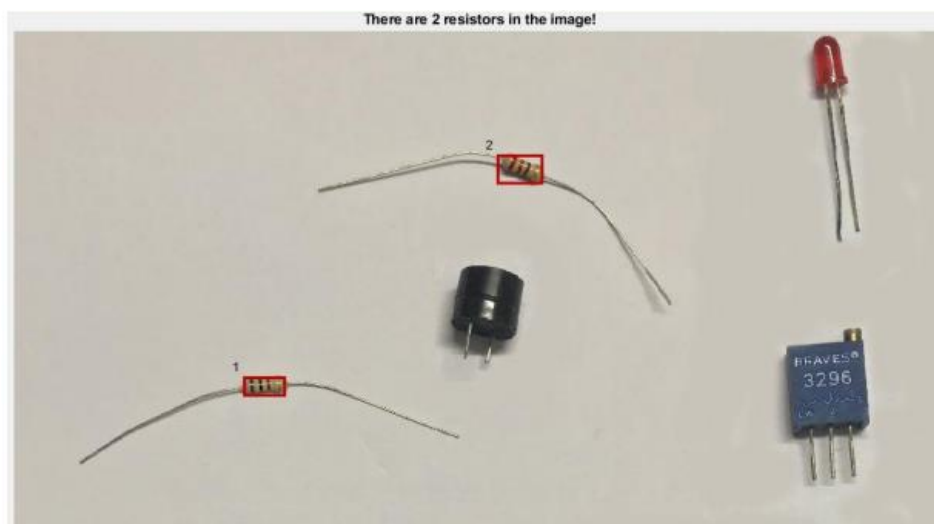


Fig: Cropping and detecting the resistors from other components:

4.4 Detecting the Color Bands:

After we have cropped the images of the resistors, we can concentrate on detecting the color bands. For that, the program has different mask functions for distinct colors. When going through these functions, a structure array is created which includes information and values of different detected bands. We have worked on 3 types of resistors which had red, black, brown, and orange color bands. So, we created mask function for these colors. The work of these functions is to compare the color range given to the function with the color band present at the resistor images and then extract the position of the bands. It will then assign 0 value to every other value which is not included in the position matrix.

Code for mask function of red:

```
% Auto-generated by colorThresholder app on 25-Feb-2024|
%-----
function [BW,maskedRGBImage] = createMaskred(RGB)

% Convert RGB image to chosen color space
I = rgb2hsv(RGB);

% Define thresholds for channel 1 based on histogram settings
channel1Min = 0.985;
channel1Max = 0.024;

% Define thresholds for channel 2 based on histogram settings
channel2Min = 0.473;
channel2Max = 0.871;

% Define thresholds for channel 3 based on histogram settings
channel3Min = 0.351;
channel3Max = 0.642;

% Create mask based on chosen histogram thresholds
sliderBW = ( (I(:,:,1) >= channel1Min) | (I(:,:,1) <= channel1Max) ) & ...
    (I(:,:,2) >= channel2Min) & (I(:,:,2) <= channel2Max) & ...
    (I(:,:,3) >= channel3Min) & (I(:,:,3) <= channel3Max);
BW = sliderBW;

% Initialize output masked image based on input image.
maskedRGBImage = RGB;

% Set background pixels where BW is false to zero.
maskedRGBImage(repmat(~BW,[1 1 3])) = 0;

end
```

Similarly, mask function was created for the mentioned colors and each resistor were passed through these functions and here is the result

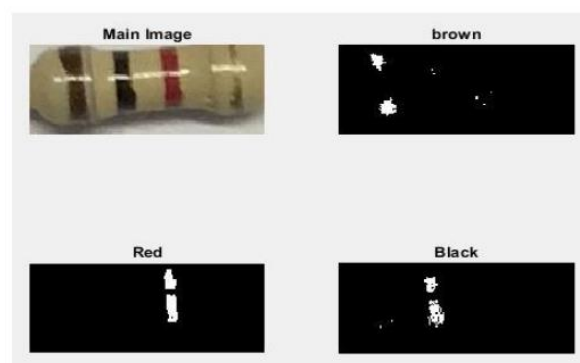
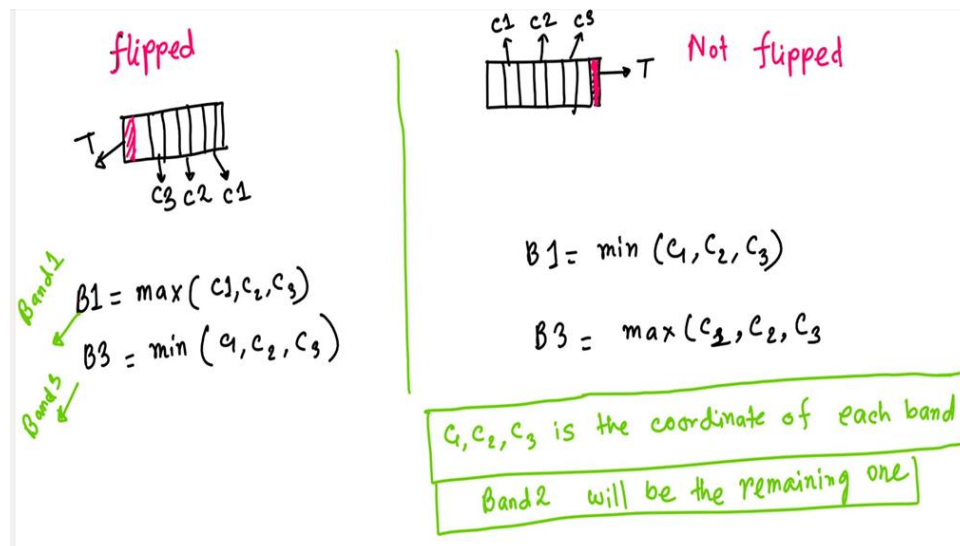


Fig: detection of color band

4.5 Resistor orientation:

In this part we checked the orientation if it is flipped or not, vertical, or horizontal. To check whether it was flip or not we used the coordinate of each color band and calculated the sum. If it is less than 250 (it was set after dealing with many cases) the sum of coordinate will be smaller in the not flipped case and sum of color band will exceed 250 in flipped case. This is explained clearly in the following figure:



Code for checking orientation:

4.5.1 Horizontal flip:

```
%% check if flipped horizontally
if bandstruct(1).BoundingBox(1,1)+bandstruct(2).BoundingBox(1,1)+bandstruct(3).BoundingBox(1,1)<250

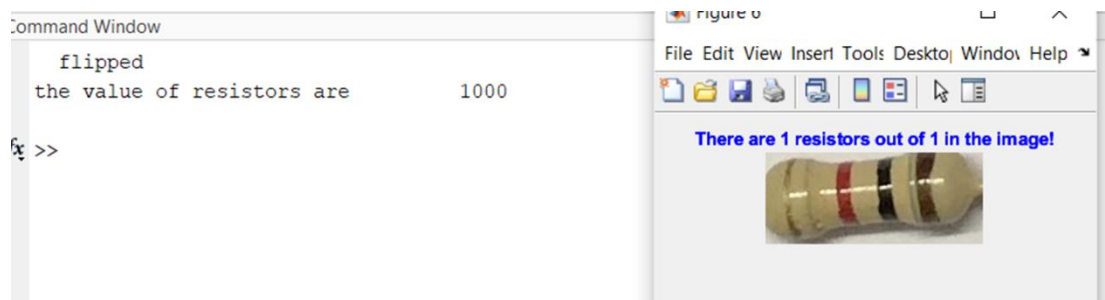
    bandnum=[ bandstruct(1).BoundingBox(1,1) bandstruct(2).BoundingBox(1,1) bandstruct(3).BoundingBox(1,1)];
    [~,I]=min(bandnum);
    Band1= bandstruct(I).Value;
    bandnum(I)=0;
    [~,I]=max(bandnum);
    Band3= bandstruct(I).Value;
    bandnum(I)=0;
else
    bandnum=[ bandstruct(1).BoundingBox(1,1) bandstruct(2).BoundingBox(1,1) bandstruct(3).BoundingBox(1,1)];
    [~,I]=min(bandnum);
    Band3= bandstruct(I).Value;
    bandnum(I)=0;
    [~,I]=max(bandnum);
    Band1= bandstruct(I).Value;
    bandnum(I)=0;
end

I=find(bandnum);
Band2= bandstruct(I).Value;
```

```
%%
disp(bandstruct(1))
disp(bandstruct(2))
disp(bandstruct(3))

sum = bandstruct(1).BoundingBox(1,1)+bandstruct(2).BoundingBox(1,1)+bandstruct(3).BoundingBox(1,1)
if sum<250
    disp(' not flipped')
else
    disp(' flipped')
end
```

Output:



4.5.2 Vertical flip:

```
elseif s(1,1)>s(1,2)
    %% check if flipped vertically
    if bandstruct(1).BoundingBox(1,2)+bandstruct(2).BoundingBox(1,2)+bandstruct(3).BoundingBox(1,2)<250
        bandnum=[ bandstruct(1).BoundingBox(1,2) bandstruct(2).BoundingBox(1,2) bandstruct(3).BoundingBox(1,2)];
        [~,I]=min(bandnum);
        Band1= bandstruct(I).Value
        bandnum(I)=0;
        [~,I]=max(bandnum);
        Band3= bandstruct(I).Value
        bandnum(I)=0;
    else
        bandnum=[ bandstruct(1).BoundingBox(1,2) bandstruct(2).BoundingBox(1,2) bandstruct(3).BoundingBox(1,2)];
        [~,I]=min(bandnum);
        Band3= bandstruct(I).Value
        bandnum(I)=0;
        [~,I]=max(bandnum);
        Band1= bandstruct(I).Value
        bandnum(I)=0;
    end
```

4.6 Calculating Value of the Resistor using Different Band Values:

After determining the corresponding color code values for Band1, Band2 and Band3, we calculate the value of resistor using,

$$\text{Resistor Value} = (\text{Band1, Band2}) \times 10^{\text{Band3}}$$

Code for resistor value calculation:

```
for r=1:length(resimg)
    run('bandfindmyv.m')
    resValue(r)= (Band1*10+Band2)*10^Band3;
end

fprintf('the value of resistors are ');
disp(resValue)
```

5 Features:

5.1 Extracting Resistors:

Our program can detect resistors from various components like sets of MOSFET, BJT, LED (light emitting diodes), Wires etc. It can extract the resistors and border them in the main picture with a red rectangular border line.

5.2 Finding Resistor bands:

Our program can detect resistor bands from the light brown color of the resistor and then extract the bounding box of the band for further calculation.

5.3 Orientation:

Our program can detect resistor bands for 4 cases. They are horizontally flipped, horizontally non-flipped, vertical flipped and vertical non-flipped.

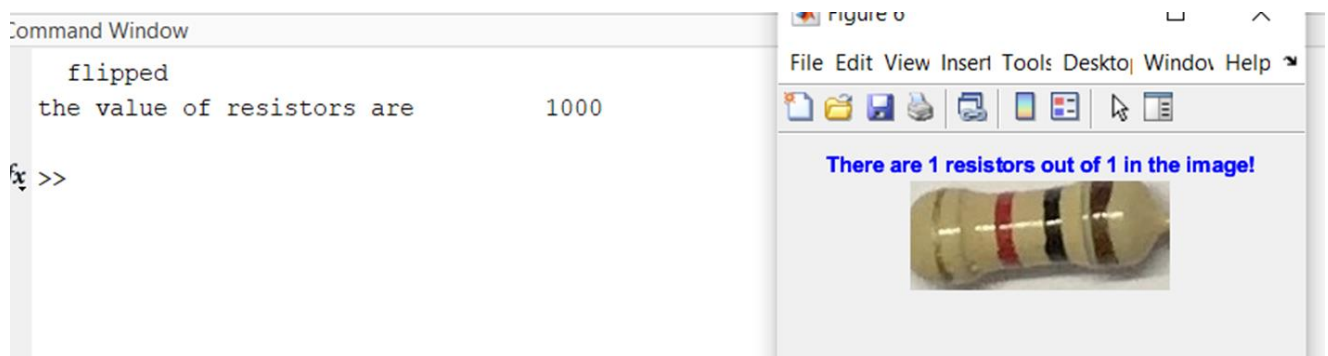
5.4 Resistor Value:

Finally, it can determine the values of the resistors according to their respected bands. It can show results even if multiple resistors are present.

6 Result:

As the project aims to detect and calculate the value of resistor from other components, here are some results:

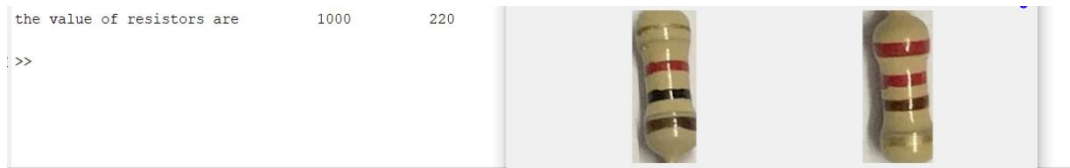
1k horizontal flipped:



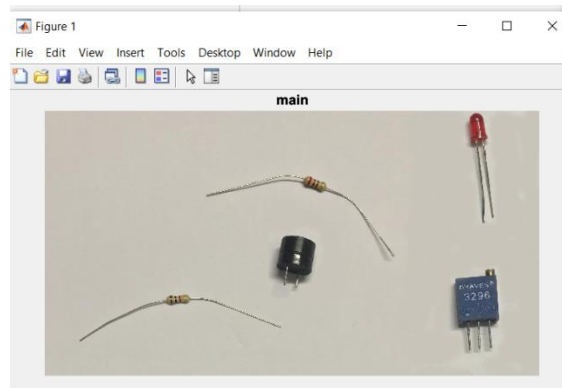
1k vertical but not flipped:



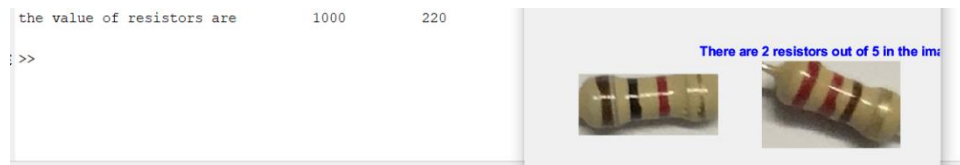
1k and 220



Multiple component:



Among them 2 of them are resistors:



7 User Manual

- Uniform white background is needed to reduce noise and proper detection.
- High quality and high-resolution camera photo will give better results.
- The photo must be taken perpendicularly to remove contrast issues.
- Shadows must be avoided for perfect detection of resistors
- This code is primarily designed for 1k, 10k, 220k resistors.

8 Novelty:

The novelty lies in utilizing image processing techniques to automatically detect the color bands of a resistor and calculate its resistance value. Unlike traditional manual methods prone to errors and time-consuming processes, this approach offers efficiency and accuracy. By leveraging computer vision algorithms, it enables real-time analysis, making it suitable for rapid component identification in various applications, such as electronics manufacturing, circuit debugging, and educational demonstrations. This innovation not only streamlines the resistor identification process but also opens avenues for integrating image processing with electronic design automation tools, enhancing overall efficiency and reliability in circuit design and analysis.

9 Social Impact:

9.1 Accessibility:

This technology can make it easier for individuals with visual impairments or color blindness to identify resistor values. By automating the process through image processing, it removes barriers to participation in electronics-related activities and education.

9.2 Education:

By providing a visual and interactive method for identifying resistor values, this technology can enhance learning experiences in electronics and engineering fields. It can be particularly beneficial in educational settings where hands-on experimentation and learning are encouraged.

9.3 Efficiency:

Traditional methods of identifying resistor values require manual inspection and interpretation of color bands, which can be time-consuming and prone to errors, especially for novices. Image processing can streamline this process, leading to faster and more accurate results.

9.4 Innovation:

By leveraging emerging technologies such as image processing, this application encourages innovation in electronics and computational imaging. It opens possibilities for further advancements in automating tasks related to component identification and characterization.

9.5 Environmental impact:

By enabling easier identification and reuse of resistors in electronic devices, this technology can contribute to reducing electronic waste. It encourages repair and repurposing of electronic components, supporting sustainability efforts.

10 Individual Contribution of Each Member

Student ID	Segment name	Description
2006142	Image segmentation and Component Detection	Algorithm for morphological operation and image segmentation
2006147	Image Segmentation and Component detection	Structuring of Components and Component isolation
2006158	Image segmentation, resistor orientation and value calculation	Image segmentation using thresholding and morphological operation, solving orientation problem like horizontal, vertical, flipped

2006161	Resistor Checking, And Band finding	Finding resistors from components. Then finding resistor bands from the resistor component and extract the bounding box segment of the band using Morphological operation
2006137	Image Segmentation and component detection	Worked on finding appropriate morphological operation and structuring element pixel value.

11 Limitations:

11.1 Accuracy

The accuracy of color detection and interpretation heavily relies on the quality of the image, lighting conditions, and the algorithm used for processing. Variations in lighting, reflections, or image noise can affect the accuracy of color detection, leading to errors in resistor value calculation. Although in our project, white background was used to increase accuracy.

11.2 Variability in Resistor Type:

While the color bands typically follow a standardized code for resistor values, there can be variations in how these codes are applied across different manufacturers or resistor types. Some resistors may have non-standard color codes or tolerance bands, which can lead to misinterpretation or incorrect value calculation.

11.3 Complexity of Image Processing Algorithms:

Developing robust image processing algorithms capable of accurately detecting and interpreting color bands can be challenging. It requires handling various scenarios, such as different resistor orientations, overlapping bands, and partial band visibility, which adds complexity to the algorithm design.

11.4 Dependency on Camera Quality

The performance of the system is heavily reliant on the quality of the camera or imaging device used to capture the resistor's image. The focusing of the image plays a key role. The band color on the resistor body must be clearly printed. Lower-quality cameras may introduce distortions, noise, or color inaccuracies, affecting the reliability of the detection and calculation process.

11.5 Calibration and Maintenance:

The system may require periodic calibration to ensure accurate color detection and value calculation. Additionally, it may need maintenance to account for changes in lighting conditions or environmental factors that could affect performance over time.

11.6 Limited resistor Field:

The code sometimes gives wrong value when it comes to the black band color. The calibration of the black color threshold value is tricky as the photo from the mobile is

not purely black. Thus, it rarely can detect the black band color. The project worked with just three resistors: 1k, 10k, 22k.

12 FUTURE WORK:

12.1 Enhanced Color Detection Algorithms:

We can develop more robust algorithms for accurately detecting and segmenting the color bands of resistors, especially in varying lighting conditions or with diverse types of resistors (e.g., carbon film, metal film, or wire-wound resistors).

12.2 Deep Learning Approaches:

We can implement deep learning techniques such as convolutional neural networks (CNNs) to automatically learn features from resistor images, potentially improving accuracy and robustness in detecting color bands.

12.3 Noise Reduction Techniques:

We can explore methods for reducing noise in images, which can improve the accuracy of color detection and subsequently the calculation of resistor values. This may involve techniques such as image denoising filters or advanced signal processing algorithms.

12.4 Integration with Mobile Applications:

We can develop a mobile application that utilizes image processing algorithms to allow users to simply take a picture of a resistor with their smartphone camera and automatically obtain its value.

12.5 User Interface Improvements:

We can design a user-friendly interface for the application, providing clear instructions for capturing images, displaying results, and potentially offering additional functionalities such as historical data tracking or resistor value conversion.

12.6 Accessibility Features:

We can ensure the application is accessible to users with disabilities by implementing features such as voice guidance or compatibility with screen readers.

13 References:

- *Automatic segmentation and classification of resistors in digital images.* (2019, October 1). IEEE Conference Publication | IEEE Xplore. <https://ieeexplore.ieee.org/stamp/stamp.jsp?arnumber=8939034>
- Y. -S. Chen and J. -Y. Wang, "Reading resistor based on image processing," 2015 International Conference on Machine Learning and Cybernetics (ICMLC), Guangzhou, China, 2015, pp. 566-571, Doi: 10.1109/ICMLC.2015.7340616.
- M. F. Demir, A. Cankirli, B. Karabatak, A. Yavariabdi, E. Mendi, and H. Kusetogullari, "Real-Time Resistor Color Code Recognition using Image Processing in Mobile Devices," 2018 International Conference on Intelligent Systems (IS), Funchal, Portugal, 2018, pp. 26-30, Doi: 10.1109/IS.2018.8710533.
- <https://www.mathworks.com/help/images/ref/colorthresholder-app.html>
- <https://www.mathworks.com/help/images/image-segmentation-using-the-color-thesholder-app.html>
- <https://www.youtube.com/watch?v=2iXvqmBbfOA&t=256s>